

Unit 1

Well-Posed Learning Problems

A **well-posed learning problem** is one where a computer program improves its performance on a given task through experience. Tom Mitchell formally defines it as:

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."

To define a learning problem properly, three key components must be identified:

1. **Task (T)** – The specific activity that the program must learn, such as playing chess, recognizing speech, or predicting stock prices.
2. **Performance Measure (P)** – A metric used to evaluate how well the program performs the task, such as accuracy, error rate, or win percentage.
3. **Training Experience (E)** – The data or interactions from which the program learns, such as past game logs, labeled speech samples, or historical stock prices.

For example:

- A checkers-playing program:
 - **Task (T):** Playing checkers.
 - **Performance Measure (P):** Percentage of games won.
 - **Training Experience (E):** Games played against itself.
- A handwriting recognition system:
 - **Task (T):** Recognizing handwritten characters.
 - **Performance Measure (P):** Percentage of correctly classified characters.
 - **Training Experience (E):** A dataset of labeled handwritten words.

A well-posed problem ensures clarity in what is being learned and how success is measured, making it essential for designing effective machine learning models.

Designing a Learning System

Designing a machine learning system involves making several key choices to ensure that the model effectively learns from experience and improves its performance over time. The design process consists of the following steps:

1. **Choosing the Training Experience**

- The type and quality of training data significantly impact learning success.
- Training experience can provide direct feedback (e.g., labeled data) or indirect feedback (e.g., rewards in reinforcement learning).
- The data should be representative of real-world scenarios where the model will be applied.

2. **Choosing the Target Function**

- The learning system must decide what it is trying to learn.

- The target function maps input features to desired outputs.
- Example: In a checkers-playing program, the target function could be a function that evaluates board states and suggests the best move.

3. Choosing a Representation for the Target Function

- The target function must be represented in a way that a computer can process efficiently.
- Common representations include:
 - Logical rules
 - Decision trees
 - Neural networks
 - Linear functions
- The choice of representation affects the model's expressiveness and computational efficiency.

4. Choosing a Function Approximation Algorithm

- The algorithm must find the best hypothesis (approximation of the target function) based on training examples.
- Learning algorithms adjust parameters to minimize errors using techniques such as:
 - Gradient Descent
 - Decision Tree Learning
 - Bayesian Learning
 - Neural Networks

5. Final System Design and Iteration

- The learning system typically consists of multiple modules:
 - **Performance System:** Executes the task (e.g., playing checkers).
 - **Critic:** Evaluates performance and provides feedback.
 - **Generalizer:** Forms general rules from examples.
 - **Experiment Generator:** Selects new training examples to improve learning.
- The system iteratively refines its hypothesis to improve performance over time.

Example: Checkers Learning System

- **Task:** Playing checkers.
- **Performance Measure:** Percentage of games won.
- **Training Experience:** Self-play (games played against itself).
- **Target Function:** A function to evaluate board states.
- **Representation:** Linear function of board features.
- **Algorithm:** Least Mean Squares (LMS) learning rule for adjusting weights.

By following these steps, a well-designed learning system can continuously improve through experience and adaptation.

Perspectives and Issues in Machine Learning

Machine learning can be viewed from different perspectives, each emphasizing a different aspect of the learning process. It also raises several fundamental issues that researchers and practitioners must address.

Perspectives on Machine Learning

1. Learning as a Search Problem

- Machine learning can be seen as searching through a vast space of possible hypotheses to find the one that best fits the observed data.
- The effectiveness of a learning algorithm depends on how efficiently it can explore this hypothesis space.
- Different learning algorithms use different strategies to search:
 - **Decision trees** search for optimal splits.
 - **Neural networks** adjust weights using gradient descent.
 - **Bayesian learning** updates probability distributions.

2. Learning as Function Approximation

- Many learning problems can be framed as finding an approximation of an unknown target function.
- The goal is to choose the best hypothesis from a hypothesis space that generalizes well to unseen data.
- Example: A handwriting recognition system approximates the function mapping images to letters.

3. Learning as Probabilistic Inference

- Some machine learning models, such as Bayesian classifiers, treat learning as a probabilistic inference problem.
- They use prior knowledge and observed data to estimate the likelihood of different hypotheses.
- This perspective is useful for tasks with uncertainty, such as medical diagnosis.

Key Issues in Machine Learning

1. What Algorithms Work Best for Different Problems?

- Different learning algorithms have varying strengths and weaknesses.
- Some models, like neural networks, perform well on complex tasks like image recognition, while simpler models like decision trees are easier to interpret.

2. How Much Data is Needed for Learning?

- The amount of training data required depends on factors such as hypothesis complexity and model flexibility.
- Theoretical bounds relate the amount of training data to confidence in the learned model.

3. How to Incorporate Prior Knowledge?

- Prior knowledge can improve learning efficiency and accuracy.
- Methods like Bayesian learning and transfer learning integrate prior information into the learning process.

4. How to Choose Training Examples?

- The selection of training examples affects the quality of the learned hypothesis.
- Active learning strategies help select the most informative examples.

5. How to Deal with Noisy and Incomplete Data?

- Real-world data is often noisy or incomplete.
- Robust learning algorithms must handle uncertainty and missing values.

6. How to Evaluate and Compare Learning Algorithms?

- Performance metrics like accuracy, precision, recall, and F1-score help evaluate models.
- Cross-validation and statistical tests compare different models.

Basic Definitions of Types of Learning

Machine learning can be categorized into different types based on how the system learns from data. The major types include:

1. Supervised Learning

- **Definition:** The learning algorithm is trained on a labeled dataset, where each input (feature) has a corresponding correct output (label).
- **Goal:** Learn a function that maps inputs to outputs correctly.
- **Examples:**
 - **Classification:** Predicts categorical labels (e.g., spam detection, digit recognition).
 - **Regression:** Predicts continuous values (e.g., house price prediction, stock market forecasting).
- **Example Algorithm:** Decision Trees, Support Vector Machines (SVM), Neural Networks, Linear Regression.

2. Unsupervised Learning

- **Definition:** The learning algorithm is given an unlabeled dataset and must find patterns, structures, or groupings in the data.
- **Goal:** Discover hidden structures without explicit supervision.
- **Examples:**
 - **Clustering:** Grouping similar objects (e.g., customer segmentation, document classification).
 - **Dimensionality Reduction:** Compressing data while retaining important features (e.g., PCA, t-SNE).

- **Example Algorithm:** K-Means Clustering, Hierarchical Clustering, Principal Component Analysis (PCA).

3. Semi-Supervised Learning

- **Definition:** A mix of supervised and unsupervised learning, where the algorithm is trained on a small amount of labeled data and a large amount of unlabeled data.
- **Goal:** Improve learning performance when labeled data is scarce.
- **Example:** Webpage classification with limited labeled examples and many unlabeled pages.
- **Example Algorithm:** Self-training, Graph-based methods.

4. Reinforcement Learning

- **Definition:** The learning system (agent) interacts with an environment, taking actions and receiving feedback (rewards or penalties).
- **Goal:** Learn an optimal strategy (policy) to maximize cumulative reward over time.
- **Examples:**
 - Robotics (e.g., training a robot to walk).
 - Game playing (e.g., AlphaGo, Chess AI).
 - Autonomous driving.
- **Example Algorithm:** Q-learning, Deep Q-Networks (DQN), Policy Gradient Methods.

5. Online vs. Offline Learning

- **Online Learning:** The model updates continuously as new data arrives (e.g., stock market predictions).
- **Offline Learning (Batch Learning):** The model is trained once on a fixed dataset and does not update until retrained.

A Concept Learning Task

Definition:

A **concept learning task** involves learning a general function or rule from specific training examples. The goal is to classify new instances correctly based on previously seen examples.

Example: EnjoySport Concept Learning Task

Consider a task where we want to predict whether a person will enjoy a sport based on weather conditions. The dataset contains attributes such as **Sky (Sunny, Cloudy, Rainy)**, **AirTemp (Warm, Cold)**, **Humidity (High, Normal)**, **Wind (Strong, Weak)**, **Water (Warm, Cool)**, and **Forecast (Same, Change)**.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Components of the Concept Learning Task:

1. **Instances (X)**: The set of all possible examples (days with different weather conditions).
2. **Target Concept (c)**: The function that labels an instance as **EnjoySport = Yes** or **No**.
3. **Hypothesis Space (H)**: The set of all possible rules that could describe the target concept.
4. **Training Examples (D)**: The observed examples with known classifications.
5. **Output Hypothesis (h)**: The best hypothesis that generalizes the training data.

Goal of Concept Learning

Find a hypothesis h in H such that $h(x) = c(x)$ for all x in X .

Example Hypothesis Representation:

A rule that generalizes positive examples might be:

$$h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$$

This means **EnjoySport = Yes** when the weather is Sunny, Warm, and Wind is Strong, but other attributes can take any value ("?" represents "any value is acceptable").

Concept Learning as Search

Definition:

Concept learning can be viewed as a **search problem**, where the goal is to find the best hypothesis that correctly classifies all training examples. The learner searches through a **hypothesis space (HHH)** to find a hypothesis that is consistent with the training data.

Key Aspects of Concept Learning as Search

1. **Hypothesis Space (H)**
 - The set of all possible hypotheses that can be considered.
 - For example, in the **EnjoySport** task, hypotheses specify rules about weather conditions.
2. **Search Strategy**
 - The learner explores the hypothesis space to find a consistent hypothesis.
 - The search can be **specific-to-general** or **general-to-specific**:
 - **Specific-to-General**: Start with the most specific hypothesis and generalize as needed.
 - **General-to-Specific**: Start with the most general hypothesis and specialize based on negative examples.
3. **General-to-Specific Ordering**
 - Hypotheses can be structured in a hierarchy where some are **more general** than others.

- Example:
 - $h_1 = (\text{Sunny, Warm, ?, Strong, ?, ?}) \rightarrow \text{More specific}$
 - $h_2 = (\text{Sunny, ?, ?, ?, ?, ?}) \rightarrow \text{More general}$
- If h_2 classifies more instances as positive than h_1 , then h_2 is **more general** than h_1

4. Search Algorithm Example: FIND-S

- **FIND-S Algorithm** starts with the most specific hypothesis and generalizes it to fit positive examples.
- Steps:
 1. Initialize h to the most specific hypothesis.
 2. For each positive training example, generalize h only as much as needed.
 3. Ignore negative examples.
 4. Return h as the final hypothesis.

Finding a Maximally Specific Hypothesis

Definition:

A **maximally specific hypothesis** is the most restrictive hypothesis that is still consistent with all positive training examples. It starts with the most specific hypothesis and generalizes only as necessary.

FIND-S Algorithm (Finding the Maximally Specific Hypothesis)

The **FIND-S algorithm** is a simple method to find the most specific hypothesis consistent with given positive examples.

Algorithm Steps:

1. **Initialize** h to the most specific hypothesis (i.e., the most restrictive, with all attributes set to " $\emptyset$ ").
2. **For each positive training example** x :
 - Compare x with h .
 - If an attribute value in h does not match x , generalize it to the observed value.
3. **Ignore negative examples.**
4. **Output** h as the final hypothesis.

Example: EnjoySport Task

Training Examples:

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Step-by-Step Execution:

1. Initialize with the most specific hypothesis:

$$h = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$$

2. First positive example (Sunny, Warm, Normal, Strong, Warm, Same):

$$h = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$$

3. Second positive example (Sunny, Warm, High, Strong, Warm, Same):

- o "Normal" $\rightarrow$ "?" (Generalize Humidity)

$$h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same})$$

4. Third example (negative) – No change.

5. Fourth positive example (Sunny, Warm, High, Strong, Cool, Change):

- o "Warm" $\rightarrow$ "?" (Generalize Water)

- o "Same" $\rightarrow$ "?" (Generalize Forecast)

$$h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$$

Final Hypothesis Output:

$$h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$$

This hypothesis is **maximally specific** because it describes the narrowest set of conditions that still covers all positive examples.

Limitations of FIND-S:

- Ignores negative examples.
- Assumes only one maximally specific hypothesis exists.
- Cannot handle noisy or inconsistent data.

Version Spaces and the Candidate Elimination Algorithm

1. Version Space

A **version space (VS)** is the subset of hypotheses in the hypothesis space H that are consistent with all training examples. It represents all possible hypotheses that could be the correct target function.

Formally,

$$VSH = \{h \in H \mid h(x) = c(x) \text{ for all training examples}\}$$

The version space is represented by two boundary sets:

- **The General Boundary G** – Contains the most general hypotheses that are still consistent.
- **The Specific Boundary S** – Contains the most specific hypotheses that are still consistent.

2. Candidate Elimination Algorithm

The **Candidate Elimination Algorithm** incrementally updates the version space as new training examples are encountered. It maintains both S and G and refines them based on training data.

Algorithm Steps:

1. Initialize S to the most specific hypothesis.
2. Initialize G to the most general hypothesis.
3. For each training example:
 - If **positive**:
 - Remove inconsistent hypotheses from G .
 - Generalize S minimally to include the example.
 - If **negative**:
 - Remove inconsistent hypotheses from S .
 - Specialize G minimally to exclude the example.
4. Repeat until all examples are processed.

Example: EnjoySport Task

Training Examples:

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Step-by-Step Execution:

Step 1: Initialization

- Specific Boundary $S \rightarrow$ Most specific hypothesis:
$$S = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$$
- General Boundary $G \rightarrow$ Most general hypothesis:
$$G = (?, ?, ?, ?, ?, ?)$$

Step 2: Process first positive example (Sunny, Warm, Normal, Strong, Warm, Same)

- Generalize S minimally:
$$S = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$$

Step 3: Process second positive example (Sunny, Warm, High, Strong, Warm, Same)

- Generalize S to cover this example:
$$S = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same})$$

Step 4: Process third negative example (Rainy, Cold, High, Strong, Warm, Change)

- Remove inconsistent hypotheses from G (still $G = (?, ?, ?, ?, ?, ?)$)
- Specialize G minimally to exclude the negative example:
$$G = (\text{Sunny}, ?, ?, ?, ?, ?), (?, \text{Warm}, ?, ?, ?, ?)$$

Step 5: Process fourth positive example (Sunny, Warm, High, Strong, Cool, Change)

- Generalize S minimally:
$$S = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$$

- Remove inconsistent hypotheses from G :

$$G = (\text{Sunny}, ?, ?, ?, ?, ?)$$

Final Output:

- Specific Boundary S :

$$(\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$$

- General Boundary G :

$$(\text{Sunny}, ?, ?, ?, ?, ?)$$

Key Insights:

1. The version space shrinks as more training examples refine S and G .
2. If $S = G$, the exact target function is found.
3. If S and G do not converge, the true concept lies somewhere between them.

Advantages of Candidate Elimination:

- Finds all possible consistent hypotheses.
- Eliminates inconsistent hypotheses incrementally.
- Provides a clear representation of uncertainty.

Limitations:

- Sensitive to noise in training data.
- Computationally expensive for large hypothesis spaces.
- Requires a well-defined hypothesis space.

Inductive Bias

Definition:

Inductive bias refers to the assumptions a learning algorithm makes to generalize beyond the observed training data. Since learning from a finite set of examples does not guarantee perfect generalization, a bias is necessary to choose between multiple hypotheses that fit the data.

Why is Inductive Bias Needed?

- Given a set of training examples, many possible hypotheses may be consistent with the data.
- Without inductive bias, a learning algorithm cannot generalize to unseen data.
- Inductive bias helps restrict the hypothesis space to make learning feasible.

Types of Inductive Bias:

1. **Restriction Bias (Hypothesis Space Bias)**
 - Limits the set of hypotheses the learner can consider.
 - Example: Decision trees limit hypotheses to tree-structured representations.
2. **Preference Bias (Search Bias)**
 - The learner considers all hypotheses but prefers some over others.

- Example: The **Occam's Razor Principle**, which prefers simpler hypotheses over complex ones.

Example: Inductive Bias in Concept Learning

Consider the **EnjoySport** task where we learn whether a person enjoys sports based on weather conditions.

- Given a few training examples, multiple hypotheses could fit the data.
- If the algorithm assumes "the simplest hypothesis is best" (Occam's Razor), it will prefer a general rule over a highly specific one.

Inductive Bias in Common Learning Algorithms:

- **Decision Trees:** Prefer shorter trees with fewer splits (Occam's Razor).
- **Neural Networks:** Regularization techniques (e.g., L2 regularization) prevent overfitting by penalizing complexity.
- **Bayesian Learning:** Uses prior probabilities to guide hypothesis selection.

Trade-offs in Inductive Bias:

- **Strong bias → Faster learning but lower flexibility** (may miss complex patterns).
- **Weak bias → More flexible but requires more data** (risk of overfitting).

Hypothesis Space and Inductive Bias

1. Hypothesis Space (HHH)

The **hypothesis space** is the set of all possible hypotheses that a learning algorithm can consider when trying to approximate a target function.

Key Aspects of Hypothesis Space:

- **Defined by the chosen representation** (e.g., decision trees, linear functions, neural networks).
- **Search space for learning algorithms**, where the goal is to find the best hypothesis that fits the training data.
- **Size and complexity** of HHH determine learning feasibility (a large hypothesis space requires more data to generalize well).

Example: Hypothesis Space in Concept Learning

For the **EnjoySport** task, where weather conditions determine whether a person enjoys sports, the hypothesis space consists of all possible rules mapping conditions to "Yes" or "No."

- If hypotheses are represented as **conjunctions of attribute constraints**, the hypothesis space is finite.
- If hypotheses are **arbitrary Boolean functions**, the space is much larger and harder to search.

2. Inductive Bias

Inductive bias refers to the assumptions a learning algorithm makes to generalize beyond training data. Since multiple hypotheses may fit the training data, bias is necessary to select a "reasonable" hypothesis.

Types of Inductive Bias:

1. **Restriction Bias (Hypothesis Space Bias):** Limits the set of hypotheses considered.
 - Example: A decision tree learner can only select hypotheses in tree form.
2. **Preference Bias (Search Bias):** Prefers some hypotheses over others.
 - Example: **Occam's Razor**, which favors simpler hypotheses over complex ones.

Relationship Between Hypothesis Space and Inductive Bias:

- A **broad hypothesis space** allows greater flexibility but requires a strong **inductive bias** to guide learning.
- A **narrow hypothesis space** has built-in **bias**, limiting the learner's ability to generalize beyond predefined structures.

Example: Learning a Linear Classifier

- **Hypothesis Space:** All linear decision boundaries.
- **Inductive Bias:** "The decision boundary is linear," meaning the model cannot learn non-linear relationships.

Evaluation and Cross-Validation

1. Evaluation of Machine Learning Models

The **evaluation** of a machine learning model measures how well it generalizes to new, unseen data. The key metrics used for evaluation depend on the type of problem:

Common Evaluation Metrics:

- **For Classification Problems:**
 - **Accuracy:** $\frac{\text{Correct Predictions}}{\text{Total Predictions}}$
 - **Precision:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
 - **Recall:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
 - **F1-score:** Harmonic mean of precision and recall.
- **For Regression Problems:**
 - **Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)**
 - **Mean Absolute Error (MAE)**
 - **R^2 Score (Coefficient of Determination)**

2. Cross-Validation

Cross-validation is a technique to assess how well a model performs by dividing the dataset into multiple parts and training/testing on different subsets.

Types of Cross-Validation:

1. Holdout Method:

- The dataset is split into **training** and **test** sets (e.g., 80% training, 20% testing).
- Simple but may not represent the overall data distribution.

2. K-Fold Cross-Validation:

- The dataset is split into **K** equal parts (folds).
- The model is trained on $K - 1$ folds and tested on the remaining fold, repeated K times.
- The average performance over all folds is taken as the final result.
- **Common choices:** $K = 5$ or $K = 10$.

3. Leave-One-Out Cross-Validation (LOOCV):

- A special case of K-Fold where K equals the number of instances in the dataset.
- Each instance is used as a test case once, and the model is trained on all remaining instances.
- Provides an unbiased estimate but is computationally expensive.

3. Importance of Cross-Validation:

- Reduces overfitting by ensuring the model performs well on different subsets of data.
- Provides a better estimate of real-world performance.
- Helps in **hyperparameter tuning**, ensuring optimal settings for model training.